

ادامه آموزش: از پیام برگشتی تا مسیر تلاش مجدد

این جزوه ادامه فایل اول است؛ مطالب مراحل ۲۸ تا ۴۹ گفت‌وگو را پوشش می‌دهد. بخش معرفی در فایل قبلی آمده بود. سؤال مرحله ۴۹ هنوز پاسخ داده نشده است. کدها بر مبنای نسخه ۷ mandatory کتابخانه RabbitMQ.Client هستند.

۱. دریافت پیام برگشتی در تولیدکننده

با mandatory: true، اگر پیام به هیچ صفی مسیریابی نشود، بروکر آن را برمی‌گرداند. رویداد زیر را قبل از انتشار ثبت می‌کنیم. برگشت پیام به‌تنهایی باعث ذخیره یا ارسال مجدد آن نمی‌شود.

```
channel.BasicReturnAsync += (sender, ea) =>
{
    var message = Encoding.UTF8.GetString(ea.Body.ToArray());
    Console.WriteLine($"Returned: {message}");
    Console.WriteLine($"Routing key: {ea.RoutingKey}");
    Console.WriteLine($"Reason: {ea.ReplyText}");
    return Task.CompletedTask;
};
```

برای مثال، اگر فقط payment.success اتصال داشته باشد، پیام payment.refund برمی‌گردد. اما اگر صف منطبق موجود باشد و مصرف‌کننده خاموش باشد، پیام در صف منتظر می‌ماند؛ mandatory روشن بودن مصرف‌کننده را بررسی نمی‌کند.

۲. تأیید انتشار از طرف بروکر

با Publisher Confirms بروکر انتشار را به تولیدکننده تأیید می‌کند. این تأیید مستقل از Consumer Ack است و به معنی اجرای موفق عملیات توسط مصرف‌کننده نیست. حتی با مصرف‌کننده خاموش نیز بروکر می‌تواند انتشار را تأیید کند.

```
var options = new CreateChannelOptions(
    publisherConfirmationsEnabled: true,
    publisherConfirmationTrackingEnabled: true);
using var channel = await connection.CreateChannelAsync(options);
```

با فعال بودن هر دو گزینه، await در BasicPublishAsync منتظر تأیید بروکر می‌ماند. رد شدن انتشار یا برگشت پیام، خطا ایجاد می‌کند. تأیید انتشار به‌تنهایی اثبات مسیریابی نیست؛ mandatory و مدیریت برگشت هم لازم‌اند.

```
try
{
    await channel.BasicPublishAsync(
        exchange: "payment.direct",
        routingKey: "payment.refund",
        mandatory: true,
        basicProperties: new BasicProperties(),
        body: Encoding.UTF8.GetBytes("Payment refunded"));
    Console.WriteLine("Publish confirmed");
}
catch (Exception ex)
{
    Console.WriteLine($"Publish failed: {ex.Message}");
}
```

در صورت خطا، خط Publish confirmed اجرا نمی‌شود. چاپ خطا خودش تلاش مجدد نیست. گرفتن عمومی Exception در این مثال آموزشی است؛ در پیاده‌سازی واقعی نوع و علت خطا باید تفکیک شود.

۳. تأیید گم‌شده و پیام تکراری

ممکن است بروکر پیام را وارد صف کرده باشد، اما تأیید به علت قطع شبکه به تولیدکننده نرسد. بنابراین دریافت نکردن تأیید، ثابت نمی‌کند پیام نرسیده است. ارسال مجدد می‌تواند نسخه تکراری بسازد.

برای هر پیام منطقی یک MessageId تعیین می‌کنیم و هنگام ارسال مجدد همان شناسه را حفظ می‌کنیم. تولید شناسه جدید در هر تلاش، تشخیص تکرار را مختل می‌کند. بروکر با تنظیم این مقدار، خودکار پیام تکراری را حذف نمی‌کند.

```
var messageId = Guid.NewGuid().ToString();
var properties = new BasicProperties { MessageId = messageId };
// Reuse properties.MessageId for retries of this message.
var receivedMessageId = ea.BasicProperties.MessageId;
```

شناسه MessageId را تولیدکننده تعیین می‌کند و برای تشخیص پیام منطقی است. شناسه DeliveryTag را بروکر برای تحویل روی کانال تعیین می‌کند و برای Ack/Nack است؛ در تحویل مجدد ممکن است تغییر کند. اگر تولیدکننده MessageId تعیین نکند، ممکن است خالی باشد.

۴. جلوگیری از اثر تکراری با جدول Inbox

مصرف‌کننده شناسه پیام‌های پردازش شده را در دیتابیس نگه می‌دارد. اگر شناسه قبلاً ثبت شده باشد، عملیات کسب‌وکار را تکرار نمی‌کند و پیام را تأیید می‌کند.

برای پیام جدید، ثبت شناسه و تغییر داده کسب‌وکار باید در یک تراکنش دیتابیس باشند. بعد از Commit موفق، Ack ارسال می‌شود. اگر برنامه بعد از Commit و قبل از Ack کرش کند، تحویل دوباره با بررسی شناسه بدون اثر تکراری مدیریت می‌شود.

اگر Ack قبل از Commit فرستاده شود و تراکنش شکست بخورد، عملیات انجام نشده ولی بروکر دیگر مسئول تحویل مجدد نیست. این الگو فقط عملیات داخل همان تراکنش را پوشش می‌دهد؛ ارسال پیامک یا

تماس خارجی خودکار با آن اتمیک نمی‌شود.

```
CREATE UNIQUE INDEX UX_Inbox_MessageId  
ON dbo.Inbox (MessageId);
```

صرف بررسی وجود شناسه کافی نیست: دو مصرف‌کننده ممکن است هم‌زمان نبودن آن را ببینند. قید یکتا اجازه ثبت دوباره نمی‌دهد. تراکنش بازنده باید کامل لغو شود تا تغییر موجودی آن نیز ثبت نشود. مثال مربوط به مصرف‌کنندگان یک سرویس با جدول مشترک است.

خطای قطع دیتابیس را نباید تکراری بودن پیام تلقی کرد. فقط پس از تشخیص معتبر تکرار و تأیید پردازش قبلی، پیام را بدون اجرای دوباره Ack می‌کنیم.

۵. خطای موقت و دائمی

برای خطای موقت مانند قطعی کوتاه دیتابیس، تلاش مجدد با فاصله و تعداد محدود مفید است. برای خطای دائمی مانند نبودن فیلد ضروری، تکرار همان داده مشکل را حل نمی‌کند و پیام باید به مسیر بررسی منتقل شود.

با `requeue: true` پیام به صف اصلی برمی‌گردد و این گزینه به‌تنهایی تأخیر یا محدودیت تلاش ایجاد نمی‌کند. با `requeue: false` پیام رد می‌شود و در صورت تنظیم مسیر پیام‌های مرده، به آن مسیر فرستاده می‌شود؛ بدون چنین مسیری ممکن است حذف شود.

۶. ساخت مسیر پیام‌های مرده

اکسچنج payment.dlx پیام را مسیریابی می‌کند و صف payment.notification.dlq آن را نگه می‌دارد. این اکسچنج، یک اکسچنج معمولی است که صف اصلی از آن برای پیام‌های مرده استفاده می‌کند.

```
await channel.ExchangeDeclareAsync(
    exchange: "payment.dlx", type: ExchangeType.Direct,
    durable: true, autoDelete: false, arguments: null);
await channel.QueueDeclareAsync(
    queue: "payment.notification.dlq", durable: true,
    exclusive: false, autoDelete: false, arguments: null);
await channel.QueueBindAsync(
    queue: "payment.notification.dlq",
    exchange: "payment.dlx", routingKey: "notification.failed");
```

روی صف اصلی، اکسچنج مقصد و کلید خروج پیام‌های مرده را تعیین می‌کنیم:

```
var arguments = new Dictionary<string, object?>
{
    ["x-dead-letter-exchange"] = "payment.dlx",
    ["x-dead-letter-routing-key"] = "notification.failed"
};
await channel.QueueDeclareAsync(
    queue: "payment.notification.v2.queue", durable: true,
    exclusive: false, autoDelete: false, arguments: arguments);
await channel.QueueBindAsync(
    queue: "payment.notification.v2.queue",
    exchange: "payment.direct", routingKey: "payment.success");
```

مصرف‌کننده را روی صف جدید ثبت می‌کنیم. نام جدید برای جلوگیری از اعلام مجدد صف قدیمی با آرگومان‌های متفاوت است؛ اختلاف تنظیم‌ها می‌تواند PRECONDITION_FAILED ایجاد کند. در محیط واقعی می‌توان تنظیم‌های قابل‌پشتیبانی را با Policy مدیریت کرد؛ صف دارای پیام را برای این تمرین حذف نکنید.

```
await channel.BasicNackAsync(
    deliveryTag: ea.DeliveryTag,
    multiple: false,
    requeue: false);
```

با این ردکردن، مسیر مورد انتظار چنین است: خروج از صف اصلی، انتشار در payment.dlx با کلید پیام به صف اصلی با requeue: true. با payment.notification.dlq و ورود به notification.failed برمی‌گردد. این مسیر برای ایمنی به تنظیم درست و دسترس‌پذیری مقصد وابسته است؛ تضمین مطلق انتقال نیست.

صف پیام‌های مرده پیام را خودکار اصلاح یا دوباره منتشر نمی‌کند. ابتدا علت شکست، داده و کد بررسی می‌شوند؛ سپس ارسال مجدد کنترل‌شده انجام می‌شود.

۷. تلاش مجدد با تأخیر و TTL

تلاش فوری و پی‌درپی می‌تواند فشار و خطای مکرر ایجاد کند. فاصله بین تلاش‌ها به دیتابیس فرصت بازیابی می‌دهد. فاصله‌های ۵، ۱۵ و ۳۰ ثانیه در چت فقط مثال بودند؛ هنوز محدودیت تعداد تلاش‌ها را پیاده نکرده‌ایم.

تنظیم x-message-ttl زمان اعتبار پیام در صف را برحسب میلی‌ثانیه تعیین می‌کند. مقدار 5000 یعنی ۵ ثانیه. پیام منقضی‌شده با مسیر پیام‌های مرده منتقل می‌شود یا در نبود آن حذف می‌شود. زمان انتقال دقیق و تضمین‌شده در حد یک تایمر نیست.

صف انتظار مصرف‌کننده ندارد؛ وگرنه پیام ممکن است قبل از انقضا مصرف شود. برای بازگشت اختصاصی به سرویس اعلان، فقط صف اصلی اعلان را با notification.retry متصل می‌کنیم:

```
await channel.QueueBindAsync(
    queue: "payment.notification.v2.queue",
    exchange: "payment.direct", routingKey: "notification.retry");
var retryArguments = new Dictionary<string, object?>
{
    ["x-message-ttl"] = 5000,
    ["x-dead-letter-exchange"] = "payment.direct",
    ["x-dead-letter-routing-key"] = "notification.retry"
};
await channel.QueueDeclareAsync(
    queue: "payment.notification.retry.5s.v2.queue",
    durable: true, exclusive: false, autoDelete: false,
    arguments: retryArguments);
```

اگر به جای کلید اختصاصی، payment.success استفاده کنیم، همه صف‌های دارای آن اتصال، از جمله گزارش‌گیری، پیام را دوباره دریافت می‌کنند. اضافه کردن اتصال notification.retry اتصال قبلی را حذف نمی‌کند؛ یک صف می‌تواند چند اتصال داشته باشد payment.success

نام صف انتظار جدید است، چون در مثال قبلی کلید خروج payment.success بود. تغییر آرگومان صف موجود با اعلام مجدد همان نام روش تغییر تنظیمات نیست.

۸. مرز پیاده‌سازی فعلی

تا اینجا خروج پیام از صف انتظار پس از انقضا به صف اصلی را تنظیم کرده‌ایم. هنوز انتقال امن پیام خطا دار از مصرف‌کننده به صف انتظار، شمارنده تلاش‌ها و سقف تلاش مجدد را پیاده نکرده‌ایم. پس کدهای این جزوه یک سامانه کامل Retry نیستند.

۹. مرور پرسش و پاسخ

پرسش: تأیید انتشار، پردازش مصرف‌کننده را هم تأیید می‌کند؟

پاسخ: خیر؛ این دو مستقل هستند.

پرسش: اگر تأیید نرسد، حتماً پیام به صف نرسیده؟

پاسخ: خیر؛ ممکن است فقط تأیید در شبکه گم شده باشد.

پرسش: برای ارسال مجدد همان پیام چه شناسه‌ای استفاده می‌کنیم؟

پاسخ: همان MessageId قبلی.

پرسش: برای پیام پردازش شده تکراری چه می‌کنیم؟

پاسخ: عملیات را تکرار نمی‌کنیم و پیام را تأیید می‌کنیم.

پرسش: چرا قید یکتا لازم است؟

پاسخ: برای جلوگیری از ثبت و اثر تکراری در پردازش هم‌زمان، همراه با تراکنش.

پرسش: برای خطای قطع دیتابیس پیام را تکراری فرض می‌کنیم؟

پاسخ: خیر؛ علت خطا باید تشخیص داده شود.

پرسش: با `requeue: true` پیام به کجا می‌رود؟

پاسخ: به همان صف اصلی.

پرسش: کدام جزء پیام شکست خورده را نگه می‌دارد؟

پاسخ: صف پیام‌های مرده؛ اکسچنج فقط مسیریابی می‌کند.

پرسش: چرا تلاش مجدد با فاصله است؟

پاسخ: برای فرصت بازیابی سرویس و کاهش تکرار سریع خطا.

پرسش: آیا اتصال جدید اتصال قبلی را حذف می‌کند؟

پاسخ: خیر؛ هر دو اتصال باقی می‌مانند. این پرسش آخر چت هنوز پاسخ داده نشده بود.

منابع برای بررسی کد و رفتار

<https://www.rabbitmq.com/client-libraries/dotnet-api-guide>

<https://www.rabbitmq.com/tutorials/tutorial-seven-dotnet>

<https://www.rabbitmq.com/docs/confirms>

<https://www.rabbitmq.com/docs/dlx>

<https://www.rabbitmq.com/docs/ttl>